Secure CAN Monitor: Progress Report

Secure CAN Monitor: Progress Report (as of 2026-11-11)

Sources: `Secure_CAN_Monitor_Plan_8_Components.md` (status board), `monitor/NEXT_STEPS.md` (handoff 2026-11-09), `COMPONENT_*` records in this folder. Last work session recorded: 2026-11-09. Nothing newer is recorded.

1. What the project is

A **detection-only** secure CAN monitor on Raspberry Pi 3 B. OP-TEE (Secure World) runs a monitor on Core 1. It checks that Linux keeps answering a heartbeat challenge and that CAN traffic keeps arriving on time. Raspberry Pi 3 has no TZASC, so there is **no prevention claim**. The work is split into 8 components. Each starts only after the previous gate passes.

2. Status at a glance

#	Component	Status	Gate
C1	Baseline and hardware readiness	**DONE** (software), 2026-11-06	Met
C2	Secure monitor foundation	**CLOSED** (software and hardware), 2026-11-09	Met (G1-G4)
C3	Core 1 integration	**COMPLETE** with recorded deviations, 2026-11-09	Met, with deviations
C4	Timing and measurement	**PARTIAL**	**Not met**
C5	Heartbeat and traffic deadline	**Software mostly done, formally "not started"**	Not checked
C6	Hardening (Milestone M1)	Not started	-
C7	Independent CAN and cross-checking	Not started (needs second board)	-
C8	Testing and final report (Milestone M2)	Not started	-

3. Component by component

C1: Baseline and hardware readiness: DONE

- Baseline saved to `SD_Card_Backups/pre-S0-sep20/` (diffs, source tarballs, SHA256SUMS, verified image and `armstub8.bin`).
- 1.1 baseline inspection: 8/8 PASS.

- 1.2 CAN controller baseline: 6/6 PASS. can0 works in loopback, restored to normal afterward. can1 is dead hardware.
- 1.3 Linux-to-Secure-World communication: 6/6. Project TA and PTA documented. `ta_roundtrip` 14/14 and `shm_roundtrip` 33/33 PASS.
- 1.4 Core 1 and FIQ baseline: 5/5. Found the critical stack-overlap problem O-01: the FIQ handler writes onto the live Core 1 stack.
- 1.5 Rollback procedure written and proven on hardware (spare card restored, `verify_boot` 16 pass, 0 fail).
- 1.6 Hardware readiness measured (Pi 3 B, GPIO map).
- 1.7 Cold reboot, `verify_boot` 18/18, and the demo passes.
- **Open:** no UART adapter, no second CAN board, no CAN-FD sender, no spare card at the start (later an 8 GB spare was used).

C2: Secure monitor foundation: CLOSED

- 2.1 State design: the `rpi3_mon` struct, the G7 (single-writer atomics) and G9 (lock only, no seqlock) rules approved.
- 2.2 Implementation: PTA commands 17-23, HMAC and keys, the Core 1 hook, and the synchronization design. Files are in optee_os `plat-rpi3`.
- Verification:
  - Host harness: 448 checks (the log lines say “448/420”; one of those two numbers is a typo that I have not resolved).
  - Mutation testing: 141 of 144 mutants caught (the 3 survivors are equivalent).
  - Static checks: 56 pass.
  - Scratch builds with the monitor on and off: 0 warnings.
- A real defect (H1) was found and fixed: a hidden `memcpy` inside a lock hold. Object-code inspection found it, where host tests could not.
- Board results: **51 PASS, 2 PARTIAL, 0 FAIL, 0 NOT RUN** out of 53 checks.
  - Passed: V2 (6,000,000/6,000,000), V14, W1, W4, W7 (forced panic, user-approved), X9, H3 (longest lock hold 7.24 µs against a 100 µs target).
  - The two PARTIAL rows are W2 (needs UART) and Y10 (needs a real CAN sender).
- 10-minute soak: 5999/5999 injected frames accepted, no lock timeouts, no starvation.
- Core 1 lock cadence set to 16384 from board data.
- **Open:** prototype keys only (not production), no real CAN-bus evidence (all frames were injected through the PTA).

C3: Core 1 integration: COMPLETE (with deviations)

- C3.1 One guarded hook in the Core 1 loop. The flag is `CFG_RPI3_CAN_MONITOR`, not the planned `_POLL`.
- C3.2 Decision D1 resolved: Option B, FIQ stays masked for the whole Core 1 loop.
- C3.3 Stack test: worst case 640 B on the normal path (2000 B including the panic path) of 4096 B. Board watermark 688 B. Headroom 3408 B against a proposed margin of 2048 B.
- C3.4 Call list: 72 functions reachable from Core 1, none forbidden.
- C3.5 Symbol diff: 3 legacy symbols dropped by D1. The plan’s “only new symbols” is not literally true.
- **Waiting on you:** confirm the 2048 B margin and accept the deviations.

C4: Timing and measurement: PARTIAL (gate NOT met)

- Done:
  - Loop model: pass 26.6 ns, lock attempt 12.6 µs, interval 448 µs at cadence 16384.
  - This corrects the earlier “75 ns/pass” figure, which included the attempt cost.
  - C4.5 CNTPCT sanity on Core 1: PASS (3000 samples over 300 s).
  - `derive_params.py` generates `mon_params.h`. Rule (a) and six timing relations pass, and the negative control fails as it should.
  - Idle and CPU latency measured.
- Blocked: floods, the CAN sender (C4.3), and the gap histogram (C4.4). There is no sender and no second board.
- Not checked: rules (b) and (c), because **R, grace and N_req are undefined**.
  - With grace = the 60 s boot grace, the `32T >= 1.25(grace+R)` rule cannot hold.
  - With N_req = 128, the 64-slot ring fails.
- Deviation: the existing TEST_STATS build replaced `CFG_RPI3_CAN_LOOP_STATS`.

C5: Heartbeat and traffic deadline: software largely done, not formally started

The status board says “not started”, but the progress file shows this:

Item	State
C5.1 Core 1 issue schedule (no burst)	Complete (host)
C5.2 cmd 22 lazy nonce, cmd 23 check order	Complete (host, 88 checks). Board: not run
C5.3 per-request deadline, k misses, boot grace, NEVER_STARTED	Complete (host). Board partial: `TIMEOUT` after 60 s with no responder
C5.4 traffic deadline, exactly-once transitions	Complete (host)
C5.5 `mon_catchup()`	Partial: behaviour verified, the function name does not exist
C5.6 FIQ mask around critical sections	Complete (covered by D1)
C5.7 acceptance T1-T25 subset	Partial: stall test added, but **the T-list is not in the repository**

The traffic half needs a CAN sender.

C6, C7, C8: not started

- C6 Hardening: refuse or log legacy commands, UART `ALIVE` line, negative tests. It also needs UART, and it is Milestone M1.
- C7 Independent CAN (chip B read by Core 1, cross-check): needs a second board and a sender.
- C8 Test campaign, soaks and `RESULTS.md`: depends on C5 and C7.

4. Earlier work (before the monitor plan, still valid)

- Real CAN hardware pipeline on can0 (loopback), `secure_can_demo` passes end to end (classify, authenticate, tamper detect, cross-core).
- OP-TEE reads the CAN chip directly over SPI0, and its interrupt reaches Secure World (Secure Peripheral Ownership Phases 1+2).

- Core 1 reserved for Secure World, verified on hardware.
- A parallel Normal/Secure World demo works.

5. What blocks progress

Decisions needed from you (none are recorded as answered): 1. Define **R**, **grace** and **N_req**. 2. Ring size: keep 64 slots or move to 128. 3. Confirm the 2048 B stack margin and accept the C3 and C4 deviations. 4. Provide a CAN sender or a second board, or accept the floods and histogram as NOT DONE. 5. Make an external copy of `SD_Card_Backups/`. It is still only on this laptop. 6. Locate the plan's T1-T25 test list. It is missing from the tree.

Hardware gaps: UART adapter, second CAN board, CAN sender.

6. Honest limits

- All monitor evidence so far uses software-injected frames, not real CAN bus traffic.
- Keys are prototype keys.
- The claim is detection only. There is no prevention.
- The Pi should be running `rpi3-sdcard-monitor-hardened-c16384.img` from the 8 GB spare. The 32 GB original card was never written. I did not re-verify the Pi's current state for this report.

7. Suggested next step

Get decisions 1-3, then close C4. After that, finish C5: run the board checks for cmd 22/23, and locate the T-list.